

CSE 4224: Digital System Design Laboratory

FPGA Based Logic Warriors: Turn-Based Combat Game

Aritra Naha Likhan

Roll: 2007092

Forhad Islam Rony

Roll: 2007065

Submitted to

Dr. Muhammad Aminul Haque Akhand

Professor

Department of Computer Science and Engineering
Khulna University of Engineering & Technology

Md. Sakhawat Hossain

Assistant Professor

Department of Computer Science and Engineering
Khulna University of Engineering & Technology



**Department of Computer Science and Engineering
Khulna University of Engineering & Technology**

1 Objective

The main objective of this project is to design and implement a real-world digital turn-based combat game using Verilog HDL on FPGA, while developing a strong understanding of hardware design concepts.

- **Pipeline-Based System Design:** Ensure proper data flow and coordination between CPU, Control Unit (CU), ALU, and Memory through a structured pipeline architecture.
- **Verilog-Based System Development:** Understand Verilog HDL and implement a system representing a real-world interactive scenario such as a player-vs-CPU combat game.
- **FPGA Implementation:** Deploy and verify the system on FPGA using push-buttons for player input and a 7-segment display for real-time output of HP, actions, and game results.
- **Rule-Based CPU AI Design:** Implement a deterministic, history-aware CPU opponent using a scoring engine, forced-override logic, and a multi-step tie-break priority chain.
- **Digital Design Fundamentals:** Apply and understand key components including CLA adder/subtractor, comparator, clamp logic, priority encoder, multiplexer, shift register, and FSM.

2 Introduction

2.1 Game Concept and Operation

Logic Warriors is a turn-based duel in which a human player faces a rule-driven CPU opponent. Each turn, both sides simultaneously choose one of four actions — Attack (A), Block (B), Heal (H), or Idle (I). The combination of both chosen actions is resolved against a fixed gain/loss table, HP values are updated, and the game continues until one side reaches zero HP.

2.2 Project Overview

This project implements a two-player digital combat game (Player vs CPU) on a Basys 3 FPGA board. Each combatant has HP ranging from 0 to 9, displayed live on the 7-segment display. The system handles player input via push-buttons, resolves combat outcomes using a lookup table, manages a rule-based CPU AI with multi-turn history tracking, and displays results in real time.

2.3 FPGA and Verilog Environment

The system is designed using Verilog Hardware Description Language (HDL), enabling parallel processing and fast computation. It is synthesized and implemented on a Xilinx Artix-7 FPGA (Basys 3 board) running at 100 MHz.

2.4 Mapping the Game to Hardware

The combat system is mapped into hardware by storing action gain/loss outcomes in a ROM lookup table and computing HP updates using a signed CLA adder with clamping. Player inputs from push-buttons are debounced and converted to single-cycle pulses. A Finite State Machine (FSM) manages the overall game flow, while a dedicated CPU AI module maintains action history and computes the CPU's next action using a scoring and tie-break engine.

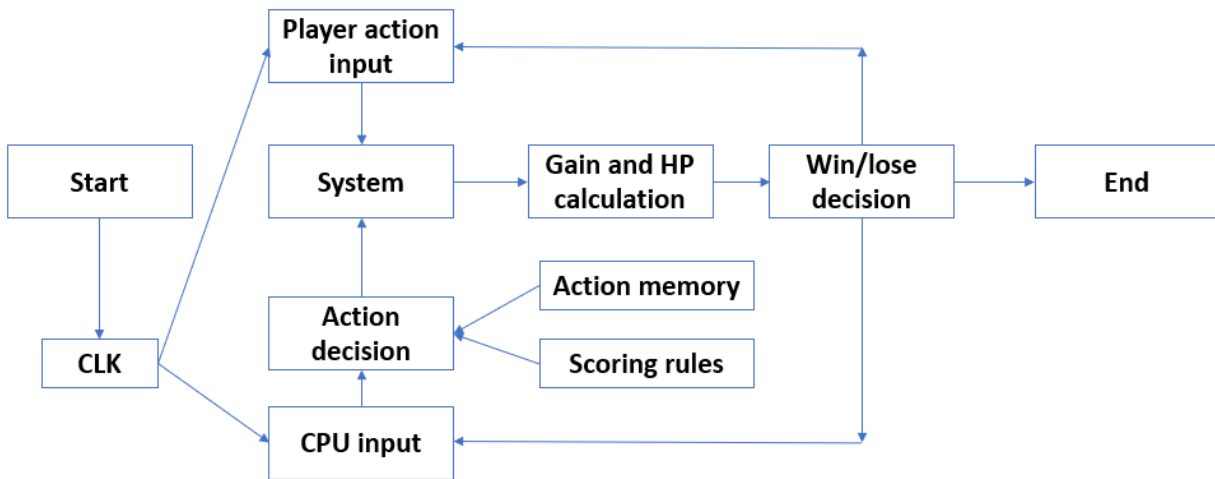


Figure-1: Game Workflow

3 System Components

3.1 Arithmetic Logic Unit (ALU)

- `cla_addsub.v`: Implements a parameterizable Carry Look-Ahead (CLA) adder/subtractor. The `add_sub` bit controls addition or subtraction using 2's complement logic. Used throughout the design for HP updates, delta computation, score accumulation, and comparisons.
- `alu_small.v`: Computes the next HP value for a combatant. It sign-extends the current HP and the signed gain, adds them with a CLA unit, clamps the result to the range $[0, 9]$, and then subtracts the original HP to produce the actual delta applied that turn.
- `cmp_n.v` / `comparator_blocks.v`: A parameterizable comparator module that generates equal (eq), less-than (lt), and greater-than (gt) outputs using a CLA subtraction. Specialized wrappers (`eq_n`, `lt_n`, `gt_n`, `le_n`, `ge_n`) expose individual comparison outputs for clean module interconnection.

The ALU data flow receives the current HP (4-bit unsigned) and the signed gain (4-bit signed) as inputs, and produces the next HP (clamped), the actual HP delta (signed), and the `is_zero` flag as outputs.

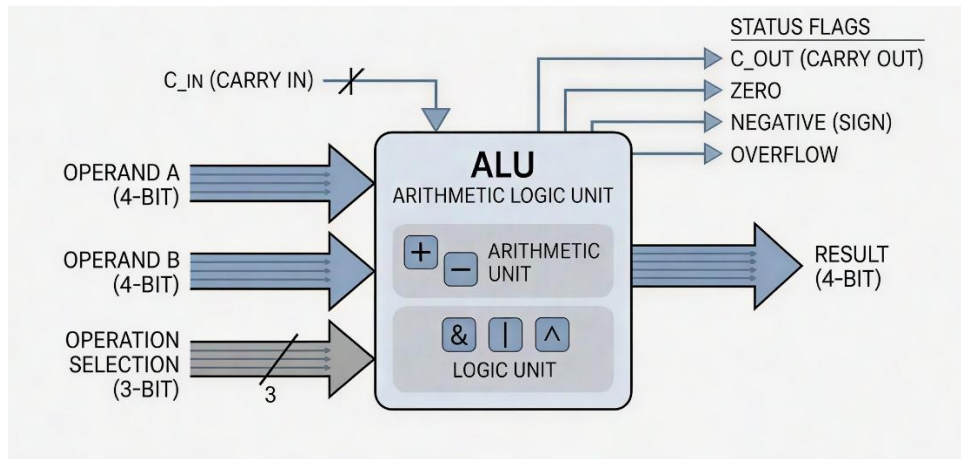


Figure-2: ALU Input-Output Block Diagram

3.2 Control Unit (CU)

- fsm_blocks.v (game_controller): Implements the main Finite State Machine (FSM) to control the entire game flow.
- Function: Manages six states — S_WAIT, S_REQ_CPU, S_SHOW_ACTIONS, S_SHOW_RESULTS, S_UPDATE, and S_GAME_OVER — cycling through each turn. When the player presses an action button while the game is running, the FSM latches the player action, requests the CPU action, shows both actions and then the results on the display for fixed time windows, updates HP, and checks for a winner.

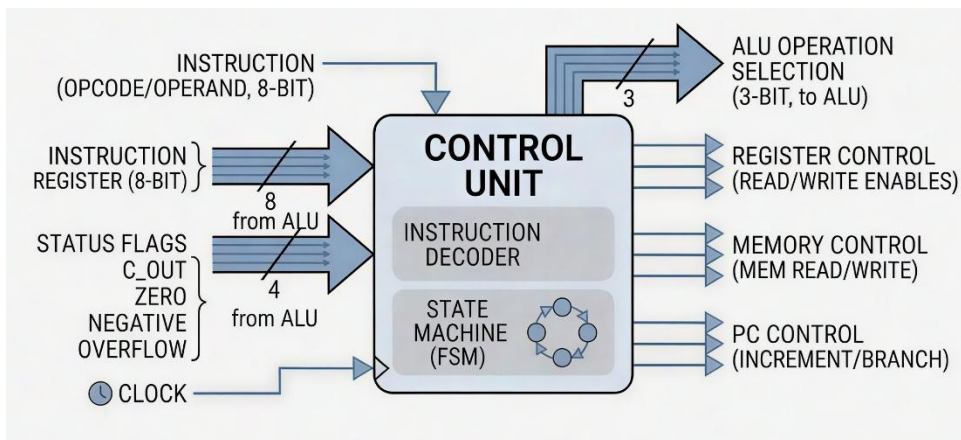


Figure-3: Control Unit Input-Output Block Diagram

The Control Unit reads player button pulses and the current game state. If a player action is valid and the game is running, it advances through the action-display-result-update pipeline. At game over, pressing BTNC restarts the match.

3.3 Memory (ROM) — Gain Lookup Table

- `gain_table.v`: Acts as a combinational ROM implementing the 4x4 action gain/loss matrix between player and CPU.
- Function: Uses combinational logic (case statements on the concatenation {`player_action`, `cpu_action`}) to map every combination of the four actions to a signed gain for each side.

The lookup table encodes 16 combat outcomes :

Table-1:

Player Action	CPU Action	Player Gain	CPU Gain
A	A	-1	-1
B	B	0	0
H	H	+2	+2
I	I	0	0
A	B	0	0
B	A	0	0
A	H	0	-1
H	A	-1	0
H	B	+2	0
B	H	0	+2
A	I	0	-2
I	A	-2	0
I	H	0	+2
H	I	+2	0
B	I	0	0
I	B	0	0

3.4 Registers (Storage)

- Registers in `game_controller`: `player_hp`, `cpu_hp`, `player_action_reg`, `cpu_action_turn`, `winner`, `display_mode`, `round_counter`, `timer_cnt` are all synchronous registers updated on the clock edge under FSM control.

- `cpu_ai_rules.v`: Maintains full history registers — P1, P2, P3 (last three player actions), C1, C2, C3 (last three CPU actions), D1, D2, D3 (last three CPU HP deltas), `player_hp_prev1`, `player_hp_prev2` — and a `forced_block_counter`. These are shifted and updated each time the UPDATE state fires.
- Scoring registers: `score_a_r`, `score_b_r`, `score_h_r`, `score_i_r`, `max_score_r`, `tie_mask_r` hold the computed action scores across clock cycles.

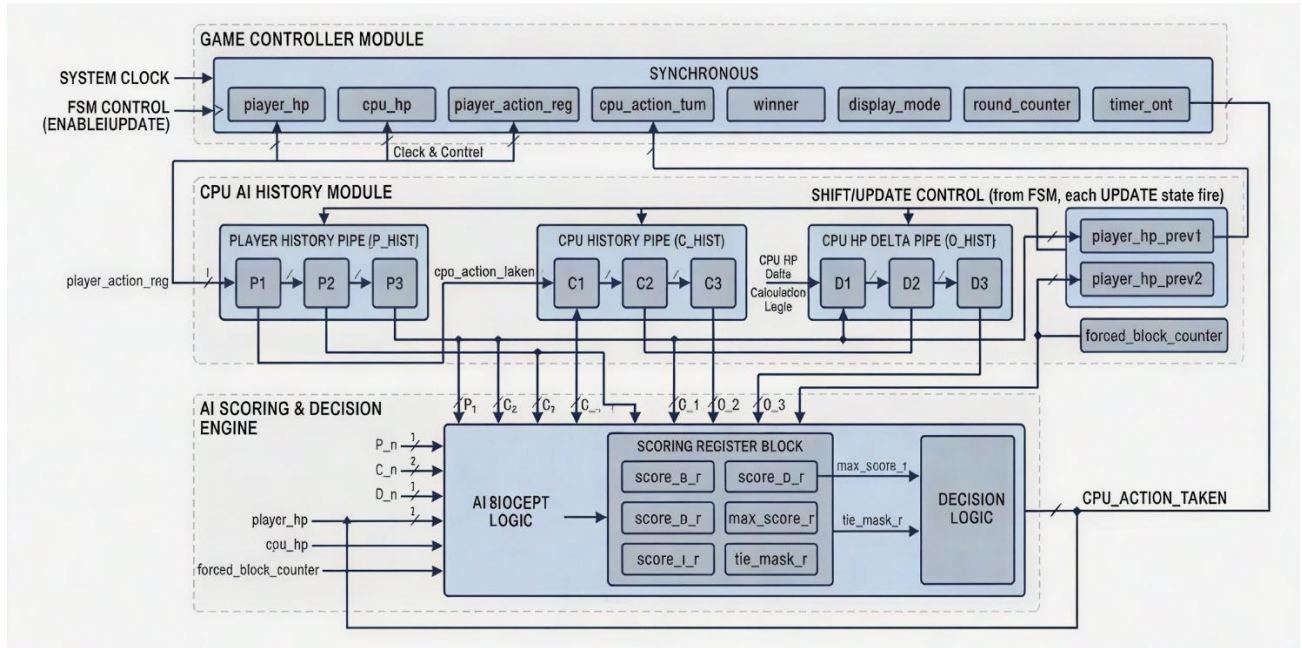


Figure-4: Register-register Datapath with Control

When a turn completes, history is shifted: $P3 \leftarrow P2$; $P2 \leftarrow P1$; $P1 \leftarrow \text{player_action_reg}$; $C3 \leftarrow C2$; $C2 \leftarrow C1$; $C1 \leftarrow \text{cpu_action_taken}$; and the signed CPU gain deltas D1/D2/D3 are updated similarly. This rolling history feeds the CPU AI scoring engine each turn.

3.5 Wires and Buses (Interconnects)

- `top.v`: Integrates all system components through named wires.
- HP buses: `player_hp [3:0]` and `cpu_hp [3:0]` carry 4-bit unsigned HP values between the game controller, ALU, and display subsystem.
- Action buses: `player_action_reg [1:0]` and `cpu_action_turn [1:0]` carry the 2-bit action codes between the controller, gain table, and display mux.
- Gain buses: `player_gain` and `cpu_gain` are signed [3:0] wires routed from the gain table through the repeat-penalty mux to the ALU and display.

3.6 Input / Output (I/O Peripherals)

- `top.v`: Handles input signals from push-buttons (including debouncing via `debounce_edge` instances) and routes them to the game controller as single-cycle pulses.
- `rom_font.v`: Stores active-low 7-segment patterns for digits 0-9 and action letters A, B, H, I, and special symbols G, O, W, L, minus, and blank.
- `display_scan.v`: Multiplexed 4-digit scan driver that cycles through the four 7-segment digits at a fixed refresh rate driven by the 100 MHz system clock.

4 Overall System Connectivity

The system is organized into four connected layers: external I/O, game controller FSM, CPU AI module, and display system.

4.1 External I/O Layer (`top.v`)

This top-level module connects the Basys 3 board with the internal game system. It instantiates five `debounce_edge` units for BTNC, BTNU, BTNL, BTNR, and BTND, then wires the resulting single-cycle pulses to the `game_controller`. `sw[15]` drives the soft reset line. LEDs show live player HP (`led[3:0]`), CPU HP (`led[7:4]`), running status (`led[8]`), and the winner flag (`led[9]`). The 7-segment display outputs (`seg`, `an`, `dp`) are driven by the `display_scan` module.

4.2 Game Controller FSM (`fsm_blocks.v`)

This module is the decision-making core of the project. It receives debounced button pulses and drives the `display_mode`, `winner`, `player_hp`, `cpu_hp`, action registers, and `round_counter`. It uses timer-based state dwell to hold the ACTION and RESULT display modes for configurable millisecond windows before advancing to the UPDATE state. BTNC toggles run/pause in all non-game-over states and restarts the game in the GAME_OVER state.

4.3 CPU AI Module (`cpu_ai_rules.v`)

This module runs continuously and exposes a `cpu_action_reg` output that the game controller latches in the S_REQ_CPU state. Internally it maintains the rolling action and delta history, computes signed scores for all four CPU actions using a chain of CLA adders, finds the maximum score via a tree of comparators and muxes, resolves ties through a deterministic priority chain, and applies the forced-block override when triggered. The player and CPU repeat-penalty flags are also output from this module and fed into the penalty mux inside the game controller.

4.4 Display System

Each valid turn result drives the display pipeline. The `display_mux` module selects what each of the four 7-segment digits shows based on the current `display_mode`: GO mode on startup, ACTION mode showing player HP / player action / CPU action / CPU HP, RESULT mode showing signed gain/loss for each side, and GAME_OVER mode showing a W or L pattern. The `display_scan` module time-multiplexes these four digit values onto the shared segment bus.

5 Element Description of the Project

The Logic Warriors system is organized into three conceptual planes: the input plane, the processing plane, and the output plane.

In the input plane, the five Basys 3 push-buttons (BTNC, BTNL, BTNR, BTNU, BTND) drive the game. Each button passes through a `debounce_edge` instance that removes mechanical bounce and emits a single rising-edge pulse per press. `sw[15]` serves as a hard synchronous reset.

In the processing plane, the `game_controller` FSM orchestrates the turn pipeline. It reads the player action from button pulses, requests the `cpu_action_reg` from the `cpu_ai_rules` module, stores both actions, then sequences through the `SHOW_ACTIONS` and `SHOW_RESULTS` states using timer-based dwell before committing the HP update. The `gain_table` provides the signed gains for the chosen action pair. The `alu_small` units compute clamped next-HP values and actual deltas. The `cpu_ai_rules` module maintains multi-turn history and outputs the CPU's next action via a scoring engine that accumulates signed weighted terms using a chain of `cla_addsub` instances.

In the output plane, the `display_mux` selects the correct digit codes for the current `display_mode`, and `display_scan` multiplexes them onto the 7-segment hardware. LEDs provide at-a-glance HP and status information without requiring the player to read the display.

6 Mathematical Operations and Execution Flow

6.1 Mathematical Operations and Formulas

The system uses hardware-efficient signed arithmetic operations throughout.

Gain Lookup (G): The gain for each player is read directly from the combinational gain table using the selected action pair.

$$G_{\text{player}}, G_{\text{cpu}} = \text{gain_table}[\text{player_action}][\text{cpu_action}]$$

Repeat Penalty (R): If a combatant has used the same action beyond its allowed streak, a penalty of -1 is applied to the gain for that turn. Attack and Block allow 3 consecutive uses; Heal allows 2. Idle has no penalty.

$G_{\text{effective}} = G - R$ where $R = 1$ if streak exceeded, else 0

HP Update (HP_{next}): The next HP is computed by adding the effective signed gain to the current HP, then clamping the result to the range [0, 9].

$HP_{\text{next}} = \text{clamp}(HP + G_{\text{effective}}, 0, 9)$

HP Delta (D): The actual change applied to HP is computed after clamping, so that history registers record the true effect rather than the unclamped gain.

$D = HP_{\text{next}} - HP$

Win Condition (W): The control unit checks after each HP update whether either HP has reached zero.

$W = (\text{player_hp} == 0) \text{ OR } (\text{cpu_hp} == 0)$

CPU Action Score (S_x): For each of the four possible CPU actions (A, B, H, I), a signed score is accumulated from weighted boolean flags derived from action history and HP values. The CPU selects the action with the highest score.

$S_A = 4 * [\text{cpu_trailing}] + 3 * [\text{p_heal2}] + 2 * [\text{p_heal_last}] + \dots$

$S_B = 5 * [\text{p_attack3}] + 3 * [\text{p_attack2}] + \dots$

$S_H = 6 * [\text{cpu_low}] + 3 * [\text{cpu_midlow}] + \dots$

$S_I = 2 * [\text{p_block2}] + 1 * [\text{p_heal_last AND cpu_ahead}] + \dots$

6.2 Execution Pipeline

The system operates in two stages.

Combinational Stage: The gain_table, comparators, scoring adder chains, and HP ALU operate continuously as combinational logic. As soon as the current action registers are stable, gains, penalties, next-HP values, deltas, and CPU scores are all updated without waiting for a clock edge.

Sequential Stage: The FSM advances through S_WAIT -> S_REQ_CPU -> S_SHOW_ACTIONS -> S_SHOW_RESULTS -> S_UPDATE. The HP registers are written only in S_UPDATE. History registers (P1/P2/P3, C1/C2/C3, D1/D2/D3) are shifted only on the update_hist_pulse that fires in S_UPDATE. The display_mode register is updated at each state transition so the display always reflects the current game phase.

6.3 CPU AI Scoring Logic

When the game moves to S_REQ_CPU, the game controller latches cpu_action_reg from the cpu_ai_rules module. The CPU decision follows a strict precedence:

- Turns 1-3: Deterministic startup — Turn 1: B; Turn 2: A; Turn 3: A if player HP dropped since two turns ago, else B.
- Forced-Block Override: If the CPU used the same non-B action for 3 consecutive turns and lost HP in each of those turns, force the next 2 CPU actions to B regardless of scores.
- Scoring Phase (Turn 4+): Compute S_A , S_B , S_H , S_I using weighted boolean flag sums. Select the action with the highest score, then apply the tie-break priority chain if scores are equal.
- Tie-Break Priority Chain: (1) If cpu_low and H is tied, choose H. (2) If $p_attack2$ and B is tied, choose B. (3) If player is passive and A is tied, choose A. (4) If p_block2 and A is tied, choose A. (5) If p_block2 and I is tied and player is not pressing, choose I. (6) Exclude most recently used tied actions if alternatives remain. (7) Rotate by round index among remaining tied actions.

6.4 Operation and Formula Table

Operation	Normal Meaning	Formula / Hardware Implementation
Gain Lookup	Map action pair to signed HP changes	$G = gain_table[player_action][cpu_action]$ (combinational case)
HP Update	Apply gain to HP with clamping	$HP_next = clamp(HP + G_effective, 0, 9)$ via $CLA + clamp_range_n$
Delta Compute	Actual HP change after clamping	$D = HP_next - HP$ via second CLA subtraction
Win Check	Detect zero HP	$is_zero = (HP_next == 0)$ via cmp_n
Repeat Penalty	Penalize over-used actions	$G_eff = G - 1$ if streak exceeded ($mux2_n$ on dec_n output)
Score Accumulation	Compute S_A , S_B , S_H , S_I	Weighted sum of boolean flags via chain of $cla_addsub \#(W(6))$
Score Comparison	Find max score, build tie mask	Tree of $gt_n + mux2_n$; tie_mask via eq_n per action
History Shift	Update rolling action/delta history	Shift registers: $P3 \leq P2$; $P2 \leq P1$; $P1 \leq player_action_reg$ (synchronous)
Round Count	Count completed turns	$round_counter \leq round_counter + 1$ via inc_n

7 Controller Design

The controller acts as the decision-making and sequencing unit of the Logic Warriors game system. It receives player button pulses and manages the flow of every game turn: latching actions, scheduling display phases with timer-based dwell, committing HP updates, and handling game-over restart.

7.1 Controller Inputs

- `clk`: system clock (100 MHz)
- `rst`: synchronous reset (`sw[15]`)
- `btnc_pulse`: center button — run/pause toggle and game-over restart
- `btntl_pulse`: left button — player selects Attack (A)
- `btmr_pulse`: right button — player selects Block (B)
- `btneu_pulse`: up button — player selects Heal (H)
- `btnd_pulse`: down button — player selects Idle (I)
- (from `cpu_ai_rules`) `cpu_action_reg [1:0]`: the CPU's chosen action for the current turn, computed by the AI module
- (from `cpu_ai_rules`) `player_repeat_penalty`, `cpu_repeat_penalty`: flags indicating whether the penalty mux should reduce the effective gain

7.2 Controller Outputs

- `display_mode [1:0]`: selects GO / ACTION / RESULT / GAME_OVER display content
- `winner`: 1 if player wins, 0 if CPU wins (valid only in GAME_OVER state)
- `player_hp [3:0]`: current player HP register
- `cpu_hp [3:0]`: current CPU HP register
- `player_action_reg [1:0]`: latched player action for the current turn
- `cpu_action_turn [1:0]`: latched CPU action for the current turn
- `player_gain` (signed [3:0]): effective gain for the player this turn (after penalty)
- `cpu_gain` (signed [3:0]): effective gain for the CPU this turn (after penalty)
- `round_counter [7:0]`: total number of completed turns
- `running`: 1 while the game is in play, 0 when paused or at game over

7.3 Controller Operation

In the WAIT state (`S_WAIT`), the controller polls for a valid player action button press while `running` is asserted. Once a press arrives, the player action is latched and the FSM moves to `S_REQ_CPU`, where the CPU's pre-computed action is captured from `cpu_ai_rules`.

In `S_SHOW_ACTIONS`, the `display_mode` is set to ACTION and a timer counts for ACTION_VIEW_MS milliseconds. After the timer expires, the FSM transitions to `S_SHOW_RESULTS`, where `display_mode` switches to RESULT and another timer holds for RESULT_VIEW_MS milliseconds.

In `S_UPDATE`, the HP registers are written with the new clamped values from the ALU, the round counter is incremented, and the `update_hist_pulse` fires to shift the history registers inside

cpu_ai_rules. If either HP reaches zero, the FSM moves to S_GAME_OVER; otherwise it returns to S_WAIT for the next turn.

In S_GAME_OVER, the display shows the win or loss pattern and the game waits for BTNC to restart.

8 Implementation

8.1 Hardware Environment and Toolchain

The system was implemented in Verilog HDL and synthesized in Xilinx Vivado for the Digilent Basys 3 FPGA board, using the Xilinx Artix-7 device xc7a35ticpg236-1l. The design operates with the board's 100 MHz system clock and uses the onboard push-buttons, slide switches, LEDs, and multiplexed 7-segment display. Pin mapping and hardware interfacing were completed through the XDC constraint file (logic_warriors_basys3.xdc).

8.2 Input Conditioning and Latching

All five push-button inputs pass through debounce_edge instances. Each debouncer uses a two-stage synchronizer followed by a counter that must reach its maximum before the output level changes; only then does a single-cycle pulse fire. This removes mechanical bounce and guarantees exactly one pulse per press regardless of how long the button is held.

When a valid action pulse arrives in S_WAIT, the selected player action is latched into player_action_reg. In S_REQ_CPU, the cpu_action_turn register is loaded from cpu_action_reg (the AI output). Both registers remain stable throughout SHOW_ACTIONS and SHOW_RESULTS so the display and gain table see consistent values.

8.3 Combinational Datapath

The main arithmetic paths are implemented as combinational logic so results are available without additional clock-cycle latency.

Gain Table Lookup: The gain_table.v module is a purely combinational block. It receives the 2-bit player_action and cpu_action and immediately returns signed 4-bit gains for both sides by decoding the 4-bit concatenated address through a case statement covering all 16 combinations.

A sample excerpt from the gain table case statement:

```
case ({player_action, cpu_action})
  {ACT_A, ACT_A}: begin player_gain = -4'sd1; cpu_gain = -4'sd1; end
  {ACT_H, ACT_B}: begin player_gain = 4'sd2; cpu_gain = 4'sd0; end
  {ACT_A, ACT_I}: begin player_gain = 4'sd0; cpu_gain = -4'sd2; end
  ...
```

endcase

Repeat Penalty Mux: The `cpu_ai_rules` module computes `player_repeat_penalty` and `cpu_repeat_penalty` flags. Each flag drives a `mux2_n` that selects between the base gain and the decremented gain (`dec_n` output) for the respective combatant.

```
mux2_n #(.W(4)) u_pgain_mux (
    .d0(player_gain_base),
    .d1(player_gain_pen),
    .sel(player_repeat_penalty),
    .y(player_gain)
);
```

HP Update ALU (`alu_small.v`): Both player and CPU have an `alu_small` instance. The module sign-extends the 4-bit unsigned HP to 5 bits and sign-extends the 4-bit signed gain to 5 bits, adds them with a `cla_addsub`, clamps the result using `clamp_range_n` to [0, 9], and subtracts the original HP from the clamped result to compute the actual delta.

```
cla_addsub #(.W(5)) u_add (
    .a(hp_ext), .b(gain_ext), .add_sub(1'b0),
    .sum(raw_sum), .cout(raw_cout), .overflow(raw_ovf)
);
clamp_range_n #(.W(5), SIGNED_MODE(1)) u_clamp (
    .in(raw_sum), .min_val(5'sd0), .max_val(5'sd9),
    .out(clamped_sum), .low_hit(lt_zero_raw), .high_hit(gt_nine_raw)
);
```

CPU Score Accumulation: Inside `cpu_ai_rules.v`, the score for each action is built as a weighted sum of boolean flags. Each term is computed by masking a constant with a replicated flag bit, then summed through a chain of 6-bit `cla_addsub` instances. For example, the score for Attack (`S_A`) is accumulated across 12 terms:

```
assign sa_t0 = {6{cpu_trailing}} & 6'sd4;
assign sa_t1 = {6{p_heal2}} & 6'sd3;
assign sa_t2 = {6{p_heal_last}} & 6'sd2;
// ... (12 terms total)
cla_addsub #(.W(6)) u_sa1 (.a(sa_t0), .b(sa_t1), .add_sub(1'b0), .sum(sa_s1), ...);
// ... chain continues through u_sa11 to produce score_a_next
```

CLA Adder/Subtractor Core (`cla_addsub.v`): The parameterizable carry look-ahead unit uses generate loops to build propagate (P) and generate (G) signals and compute carry in parallel for any width `W`.

```
assign bx = b ^ {W{add_sub}};
assign c[0] = add_sub;
```

```

generate
  for (i = 0; i < W; i = i + 1) begin : GEN_CLA
    assign p[i]  = a[i] ^ bx[i];
    assign g[i]  = a[i] & bx[i];
    assign c[i+1] = g[i] | (p[i] & c[i]);
    assign sum[i] = p[i] ^ c[i];
  end
endgenerate
assign overflow = c[W] ^ c[W-1];

```

Score Comparison and Tie Resolution: After all four scores are computed, a tree of `gt_n` and `mux2_n` modules finds the maximum score. `eq_n` modules then compare each score against the maximum to produce a 4-bit `tie_mask`. The `priority_select4`, `exclude_if_alternative4`, and `rotate_priority_select4` modules implement the deterministic 7-step tie-break chain.

8.4 Control Unit and Synchronization

The `game_controller` module is a synchronous FSM coded in a single always `@(posedge clk)` block. The state register is updated each clock edge based on the current state, button pulses, timer values, and the running flag. State-encoding uses simple 3-bit binary constants (`S_WAIT = 3'd0` through `S_GAME_OVER = 3'd5`), and a `decoder3to8` module produces one-hot state signals for clean combinational guard expressions.

Timer dwell is implemented with a 32-bit counter (`timer_cnt`) that increments each clock cycle via an `inc_n` instance. Comparator `eq_n` units check whether `timer_cnt` equals the pre-computed `ACTION_LAST` or `RESULT_LAST` constants (derived from `CLK_HZ` and the millisecond parameters).

8.5 CPU AI History and Storage

The `cpu_ai_rules` module uses synchronous registers for all history and scoring state. On reset or `restart_pulse`, all history registers (`P1/P2/P3`, `C1/C2/C3`, `D1/D2/D3`) are initialized to neutral values and all score registers are zeroed. On `update_hist_pulse` (which fires in `S_UPDATE`), the pipeline shifts: the oldest entry is discarded, all others advance by one position, and the current turn's action and delta are written into `P1` and `D1` respectively.

The `forced_block_counter` is a 2-bit register: it is set to 2 when the trigger condition fires, decremented each turn while non-zero, and cleared to 0 on restart. The `cpu_turn_index` counts CPU decisions from 0 to 7 (wrapping) to track the deterministic startup phase and provide the rotation index for tie-breaking.

8.6 Display Multiplexing and User Interface

The display system is implemented using a multiplexed 7-segment scan driver (display_scan.v) and a display mode selector (display_mux.v).

In MODE_GO (power-on / paused before round 1), the display shows 'G' on digit 3 and 'O' on digit 2.

In MODE_ACTION, digit 3 shows player HP (0-9), digit 2 shows the player's action letter (A/B/H/I), digit 1 shows the CPU's action letter, and digit 0 shows CPU HP.

In MODE_RESULT, digit 3 shows the sign of the player's gain (minus symbol or blank), digit 2 shows the magnitude, digit 1 shows the sign of the CPU's gain, and digit 0 shows the CPU's magnitude.

In MODE_GAME_OVER, the display shows a 'W'-like three-segment pattern if the player won, or an 'L' pattern if the player lost. The rom_font.v module stores all segment patterns as active-low 8-bit masks.

8.7 Slide Switches

Switch	Function
sw[15]	Soft reset (active high; resets all registers to initial state)
sw[14:0]	Unused (available for future extensions)

8.8 Push-Buttons

Button	Function
BTNC	Run/Pause toggle; Restart game when in GAME_OVER state
BTNL	Player selects Attack (A)
BTNR	Player selects Block (B)
BTNU	Player selects Heal (H)
BTND	Player selects Idle (I)

8.9 LEDs

LED	Function
led[3:0]	Player HP (4-bit binary, live)
led[7:4]	CPU HP (4-bit binary, live)
led[8]	Running flag (1 = game in progress)

led[9]	Winner flag (1 = player won, valid at game over)
led[15:10]	Unused (tied to 0)

8.10 Project File Hierarchy (Module Instantiation Tree)

The following hierarchy shows how the Verilog modules are connected in the project:

```

top.v (Physical Hardware Wrapper for Basys 3 board)
|
|-- debounce_edge (5 instances: cleans noisy push-button inputs)
|
|-- game_controller (Core game FSM — inside fsm_blocks.v)
| |
| | |-- decoder3to8      (state one-hot decode)
| | |-- priority_select4 (player action arbitration)
| | |-- gain_table      (4x4 action gain/loss lookup)
| | |-- alu_small x2     (player HP ALU + CPU HP ALU)
| | | |-- cla_addsub     (parameterizable CLA add/sub)
| | | |-- 'clamp_range_n (clamp to [0,9])
| | |-- cpu_ai_rules     (rule-based CPU action selector)
| | | |-- history_valid_counter
| | | |-- eq_n / lt_n / gt_n / le_n / ge_n (comparators)
| | | |-- cla_addsub chains (score accumulators)
| | | |-- priority_select4 / rotate_priority_select4
| | | |-- 'exclude_if_alternative4
| | |-- inc_n (timer and round counter increment)
| | |-- 'eq_n (timer done comparators)
|
|-- display_mux (selects digit content by display_mode)
| |-- rom_font x4 (segment patterns for each digit)
|
'-- display_scan (4-digit multiplexed scan driver)

```


9 State Diagram

The game FSM has two top-level modes selected by `sw[15]`: when `sw[15] = 0` the normal game flow is active; when `sw[15] = 1` the system is held in synchronous reset and all registers return to initial values.

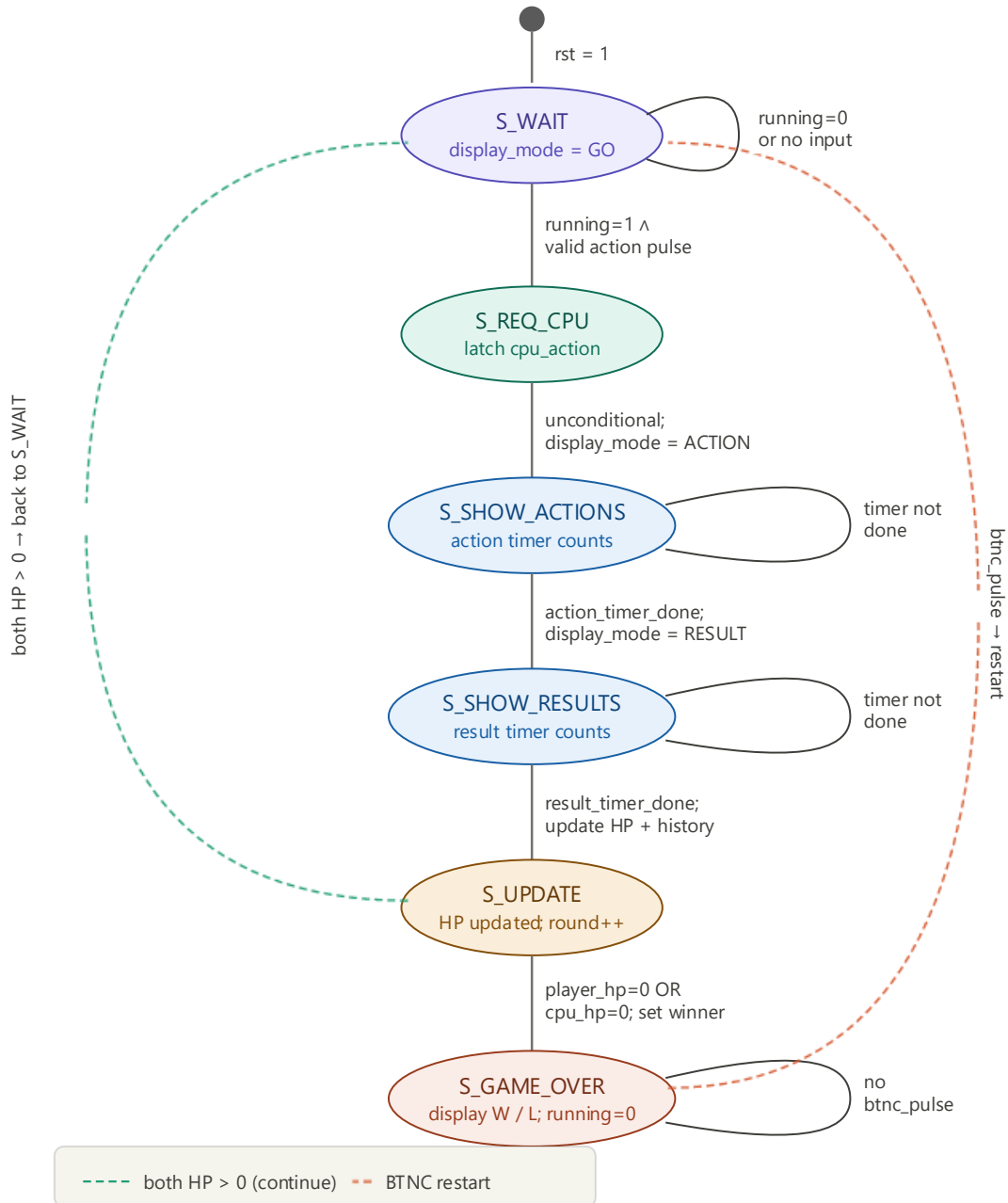


Figure-5: State Diagram of Logic Warriors

The six FSM states and their transitions are described in the tables below.

9.1 Player Mode State Table

Present State	Input / Condition	Next State	Output / Action
S_WAIT	rst = 1	S_WAIT (reset)	All registers initialized; display_mode = GO
S_WAIT	running = 0 AND round = 0	S_WAIT	display_mode = GO; timer reset
S_WAIT	running = 1 AND valid action pulse	S_REQ_CPU	player_action_reg <= chosen action
S_REQ_CPU	—	S_SHOW_ACTIONS	cpu_action_turn <= cpu_action_reg; display_mode = ACTION; timer reset
S_SHOW_ACTIONS	running = 1 AND action_timer_done	S_SHOW_RESULTS	timer reset
S_SHOW_ACTIONS	running = 1 AND timer not done	S_SHOW_ACTIONS	timer increments
S_SHOW_RESULTS	running = 1 AND result_timer_done	S_UPDATE	display_mode = RESULT; timer reset
S_SHOW_RESULTS	running = 1 AND timer not done	S_SHOW_RESULTS	timer increments
S_UPDATE	player_hp_next = 0 OR cpu_hp_next = 0	S_GAME_OVER	HP updated; round++; history shifted; winner set; display_mode = GAME_OVER; running = 0
S_UPDATE	both HP > 0	S_WAIT	HP updated; round++; history shifted; display_mode = ACTION
S_GAME_OVER	btnc_pulse	S_WAIT (restart)	All registers re-initialized; display_mode = GO
S_GAME_OVER	—	S_GAME_OVER	display_mode = GAME_OVER; inputs ignored except BTNC

9.2 CPU AI State Table

The cpu_ai_rules module does not have its own FSM states; it runs as a combinational + register block whose outputs update continuously. However, the CPU decision follows a strict internal priority sequence each time it is queried (in S_REQ_CPU):

Condition	CPU Action Selected	Notes
forced_block_counter > 0	B (BLOCK)	Overrides all other logic; counter decremented after turn
cpu_turn_index = 0	B (BLOCK)	First CPU turn always blocks
cpu_turn_index = 1	A (ATTACK)	Second CPU turn always attacks
cpu_turn_index = 2	A if player HP fell, else B	Third turn: HP-comparison rule
Unique max score (one action wins)	That action	Scoring phase; turns 4 onward
Tie: cpu_low AND H tied	H (HEAL)	First tie-break priority
Tie: p_attack2 AND B tied	B (BLOCK)	Second tie-break priority
Tie: player passive AND A tied	A (ATTACK)	Third tie-break priority
Tie: p_block2 AND A tied	A (ATTACK)	Fourth tie-break priority
Tie: p_block2 AND I tied AND not pressing	I (IDLE)	Fifth tie-break priority
Tie: alternatives after excluding recent actions	First remaining tied	Sixth tie-break priority
Remaining tie	Rotate by cpu_turn_index[1:0]	Final fallback rotation

10 Conclusion

This project successfully implemented a real-time FPGA-based turn-based combat game using Verilog HDL on the Basys 3 board. The design integrated a parameterizable CLA adder/subtractor arithmetic core, a signed HP-update ALU with clamping, a combinational gain lookup table, a rule-based CPU AI engine with multi-turn history tracking and a weighted scoring system, and a multiplexed 7-segment display pipeline, all coordinated by a synchronous FSM controller.

The project provided practical experience with fundamental digital design concepts including carry look-ahead arithmetic, signed number representation and clamping, priority encoding, multiplexed display driving, debounce circuit design, and FSM-based system control. The CPU AI module in particular demonstrated how complex decision logic — including scoring, tie-breaking, override conditions, and history-dependent behavior — can be implemented entirely with combinational blocks and synchronous registers without any software or randomness.

Overall, the project demonstrates how digital system design concepts can be combined to model a real-world interactive embedded application on reconfigurable hardware.

11 References

- M. M. Mano, Digital Logic and Computer Design.
- Course lecture slides and lab instructions provided in the Digital System Design Laboratory.
- Digilent Basys 3 Reference Manual, Digilent Inc.
- Xilinx Vivado Design Suite User Guide, Xilinx Inc.
- Verilog HDL: A Guide to Digital Design and Synthesis, S. Palnitkar.